Bias in Machine Learning Software: Why? How? What to Do?

Joymallya Chakraborty
jchakra@ncsu.edu
North Carolina State University
Raleigh, USA

Suvodeep Majumder
smajumd3@ncsu.edu
North Carolina State University
Raleigh, USA

Tim Menzies
timm@ieee.org
North Carolina State University
Raleigh, USA

arXiv:2105.12195v3 [cs.LG] 9 Jul 2021

ABSTRACT

Increasingly, software is making autonomous decisions in case of criminal sentencing, approving credit cards, hiring employees, and so on. Some of these decisions show bias and adversely affect certain social groups (e.g. those defined by sex, race, age, marital status). Many prior works on bias mitigation take the following form: change the data or learners in multiple ways, then see if any of that improves fairness. Perhaps a better approach is to postulate root causes of bias and then applying some resolution strategy.

This paper checks if the root causes of bias are the prior decisions about (a) what data was selected and (b) the labels assigned to those examples. Our Fair-SMOTE algorithm removes biased labels; and rebalances internal distributions so that, based on sensitive attribute, examples are equal in positive and negative classes. On testing, this method was just as effective at reducing bias as prior approaches. Further, models generated via Fair-SMOTE achieve higher performance (measured in terms of recall and F1) than other state-of-the-art fairness improvement algorithms.

To the best of our knowledge, measured in terms of number of analyzed learners and datasets, this study is one of the largest studies on bias mitigation yet presented in the literature.

CCS CONCEPTS

• **Software and its engineering → Software creation and management**; • **Computing methodologies → Machine learning**.

KEYWORDS

Software Fairness, Fairness Metrics, Bias Mitigation

ACM Reference Format:
Joymallya Chakraborty, Suvodeep Majumder, and Tim Menzies. 2021. Bias in Machine Learning Software: Why? How? What to Do?. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21), August 23–28, 2021, Athens, Greece.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3468264.3468537

1 INTRODUCTION

It is the ethical duty of software researchers and engineers to produce quality software that makes fair decisions, especially for high-stake software that makes important decisions about human lives.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '21, August 23–28, 2021, Athens, Greece
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8562-6/21/08…$15.00
https://doi.org/10.1145/3468264.3468537

Sadly, there are too many examples of machine learning software exhibiting unfair/biased behavior based on some protected attributes like sex, race, age, marital status. For example:

- Amazon had to scrap an automated recruiting tool as it was found to be biased against women [1].
- A widely used face recognition software was found to be biased against dark-skinned women [2].
- Google Translate, the most popular translation engine in the world, shows gender bias. "She is an engineer, He is a nurse" is translated into Turkish and then again into English becomes "He is an engineer, She is a nurse" [3].

Prior works on managing fairness in machine learning software have tried mitigating bias by exploring a very wide space of control parameters for machine learners. For example, Johnson et al. [4] executed some (very long) grid search that looped over the set of control parameters of a learner to find settings that reduced bias. Another approach, published in FSE'20 by Chakraborty et al. [5] used stochastic sampling to explore the space of control options (using some incremental feedback operator to adjust where to search next). While these approaches were certainly useful in the domains tested by Johnson and Chakraborty et al., these methods are "dumb" in a way because they do not take advantage of domain knowledge. This is a problem since, as shown by this paper, such domain knowledge can lead to a more direct (and faster and more effective) bias mitigation strategy.

The insight we offer here is that the root causes of bias might be the *prior decisions that generated the training data.* Those prior decisions affect (a) what data was collected and (b) the labels assigned to those examples. Hence, to fix bias, it might suffice to apply mutators to the training data in order to

> *(A) remove biased labels; and (B) rebalance internal distributions such that they are equal based on class and sensitive attributes.*

This "Fair-SMOTE" tactic uses *situation testing* [6, 7] to find biased labels. Also, it balances frequencies of sensitive attributes and class labels (whereas the older SMOTE algorithm [8] just balances the class labels). We recommend Fair-SMOTE since:

- Fair-SMOTE is *faster* (220%) than prior methods such as Chakraborty et al.'s [5] method.
- Models generated via Fair-SMOTE are more *effective* (measured in terms of recall and F1) than Chakraborty et al. [5] and another state-of-the-art bias mitigation algorithm [9].

Overall, this paper makes the following contributions:

- We demonstrate two main reasons for the training data being biased - a) data imbalance, b) improper data label.
- We combine finding bias, explaining bias, and removing bias.
- We show that traditional class balancing techniques damage the fairness of the model.

Figure 1: Many tools try to find or explain or mitigate bias. Fair-SMOTE addresses all three problems, at the same time.

- Prior works compromised performance while achieving fairness. We achieve fairness with better recall and F1 score.
- To the best of our knowledge, this study explores more learners and datasets than prior works that were based on just a handful of datasets and/or learners (e.g. [9–11]).

Before beginning, we digress to clarify two points. Firstly, in this paper, *training data* is mutated by Fair-SMOTE but the *test data* remains in its original form.

Secondly, one danger with mutating data is that important associations between variables can be lost. Hence, in this work, we take care to mutate by extrapolating between the values seen in two neighboring examples. In that mutation process, Fair-SMOTE *extrapolates all the variables by the same amount*. That is, if there exists some average case association between the pre-mutated examples, then that association is preserved in the mutant. To facilitate open science, the source code and datasets used in this study are all available online[1].

2 RELATED WORK

Fairness in ML model is a well-explored research topic in the ML Community. Recently, the software engineering community has also become interested in the problem of fairness. Software researchers from UMass Amherst have developed a python toolkit called Fairkit to evaluate ML models based on various fairness metrics [4]. ICSE 2018 and ASE 2019 conducted separate workshops for software fairness [12, 13]. Big software industries have started taking this fairness problem seriously as well. IBM has created a public GitHub repository AI Fairness 360 [14] where most popular fairness metrics, and mitigation algorithms can be found. Microsoft has created a dedicated research group for fairness, accountability, transparency, and ethics in AI [15]. Facebook has developed a tool to detect bias in their internal software [16].
This section reviews the fairness literature in order to find two baseline systems (which we will use to evaluate Fair-SMOTE).

Our first baseline is Chakraborty et al. [5] from FSE'20 that viewed bias mitigation as a multiple-goal hyperparameter optimization problem. Their stochastic search found learner control parameters that reduced prediction bias. We prefer Chakraborty et al.'s method over the grid search of Johnson et al. [4] since grid search (a) can be very slow and (b) it is strongly deprecated in the machine learning literature [17]. That said, one drawback with the method of Chakraborty et al. is that it did not guide its search via domain knowledge (such as those listed in the introduction).

For a second baseline system, we reviewed popular bias mitigation works including Reweighing [30], Prejudice Remover Regularizer [31], Adversarial debiasing [10], Equality of Opportunity [32], and Reject option classification [11]. While all these tools are useful for mitigating bias, as a side effect of *improving* fairness these methods also *degrade* learner performance (as measured by accuracy & F1 [9–11, 30–32]). Hence, a truism in the research community is that, as said by Berk et al. [33]:

> *It is impossible to achieve fairness and high performance simultaneously (except in trivial cases).*

In 2020, Maity et al. tried to reverse the Berk et al. conclusion. To do so they had to make the unlikely assumption that the test data was unbiased [34]. Our experience is that this assumption is difficult to achieve. Also, based on our experience with Fair-SMOTE (described below), we argue that this assumption may not even be necessary. We show below that it is possible to mitigate bias while maintaining/improving performance at the same time. The trick is how the data is mutated. Fair-SMOTE *extrapolates all the variables by the same amount*. In this way, associations between variables (in the pre-mutated examples), can be preserved in the mutant.

As to other work, a recent paper from Google [35] talked about data labels being biased. This motivated us to look into data labels. Yan et al. commented that under representation of any sensitive group in the training data may give too little information to the classifier resulting bias [36]. This work inspired us to look into data distribution and class distribution to find root causes for bias.

From the remaining literature, we would characterize much of it as looking *vertically* down a few columns of data (while in our approach, our mutators work *horizontally* across rows of data). Further, much prior works only *test* for bias (whereas in our approach, we propose how to *fix* that bias).

There are many examples of "vertical" studies that just test for bias. Zhang et al. [37] built a directed acyclic graph (DAG) containing cause-effect relations of all the attributes to see which features directly or indirectly affect the outcome of the model. Loftus et al. [38] show methods to detect if sensitive attributes such as race, gender are implicated in model outcome. THEMIS [39] randomly perturbs each attribute to see whether model discriminates amongst individuals. AEQUITAS [40], uses random, semi-directed, and fully-directed generation techniques that make it more efficient than THEMIS. Recently, IBM researchers have developed a new testing technique by combining LIME (Local Interpretable Model

[1]https://github.com/joymallyac/Fair-SMOTE

Table 1: Description of the datasets used in the experiment.

Dataset	#Rows	#Features	Protected Attribute	Description
Adult Census Income [18]	48,842	14	Sex, Race	Individual information from 1994 U.S. census. Goal is predicting income >$50,000.
Compas [19]	7,214	28	Sex,Race	Contains criminal history of defendants. Goal predicting re-offending in future
German Credit [20]	1,000	20	Sex	Personal information about individuals & predicts good or bad credit.
Default Credit [21]	30,000	23	Sex	Customer information for people from Taiwan. Goal is predicting default payment.
Heart Health [22]	297	14	Age	Patient information from Cleveland DB. Goal is predicting heart disease.
Bank Marketing [23]	45,211	16	Age	Contains marketing data of a Portuguese bank. Goal predicting term deposit.
Home Credit [24]	37,511	240	Sex	Loan applications for individuals. Goal is predicting application accept/reject.
Student Performance [25]	1,044	33	Sex	Student achievement of two Portuguese schools. Target is final year grade.
MEPS-15,16 [26]	35,428	1,831	Race	Surveys of families, individuals, medical providers, employers. Target is "Utilization".

Table 2: Definition of the performance and fairness metrics used in this study.

Performance Metric	Ideal Value	Fairness Metric	Ideal Value
Recall = TP/P = TP/(TP+FN)	1	**Average Odds Difference (AOD)**: Average of difference in False Positive Rates(FPR) and True Positive Rates(TPR) for unprivileged and privileged groups [27]. TPR = TP/(TP + FN), FPR = FP/(FP + TN), $AOD = [(FPR_U - FPR_P) + (TPR_U - TPR_P)] * 0.5$	0
False alarm = FP/N = FP/(FP+TN)	0	**Equal Opportunity Difference (EOD)**: Difference of True Positive Rates(TPR) for unprivileged and privileged groups [27]. $EOD = TPR_U - TPR_P$	0
Accuracy = $\frac{(TP+TN)}{(TP+FP+TN+FN)}$	1	**Statistical Parity Difference (SPD)**: Difference between probability of unprivileged group (protected attribute PA = 0) gets favorable prediction ($\hat{Y} = 1$) & probability of privileged group (protected attribute PA = 1) gets favorable prediction ($\hat{Y} = 1$) [28]. $SPD = P[\hat{Y} = 1 \mid PA = 0] - P[\hat{Y} = 1 \mid PA = 1]$	0
Precision = TP/(TP+FP)	1	**Disparate Impact (DI)**: Similar to SPD but instead of the difference of probabilities, the ratio is measured [29]. $DI = P[\hat{Y} = 1 \mid PA = 0]/P[\hat{Y} = 1 \mid PA = 1]$	1
F1 Score = $\frac{2 * (Precision * Recall)}{(Precision + Recall)}$	1		

Explanation) and symbolic execution [41]. Zhang et al. developed a fairness testing method that is applicable for DNN models [42] (they combined gradient computation and clustering techniques).

Finally, we can find one paper that seems like a competitive technology to the methods of this paper. Optimized Pre-processing (OP) by Calmon et al. [9] tries to find, then fix, bias. OP is a data-driven optimization framework for probabilistically transforming data in order to reduce algorithmic discrimination. OP treats bias mitigation as a convex optimization problem with goals to preserve performance and achieve fairness. We chose this work as a baseline because, like Fair-SMOTE, it is also a pre-processing strategy.

In summary, prior work suffered from

- Some methods only find bias, without trying to fix it.
- Some methods for fixing bias have an undesired side effect: leaner performance was degraded.

For the rest of this paper, we explore a solution that finds root causes of bias, and directly implement mitigation of those causes (resulting in less bias and better performance than seen in prior work).

3 WHAT ARE THE ROOT CAUSES OF BIAS?

To understand what might cause bias, at first, we need some terminology. Table 1 contains 10 datasets used in this study. All of them are binary classification problems where the target class has only two values.

- A class label is called a *favorable label* if it gives an advantage to the receiver such as receiving a loan, being hired for a job.
- A *protected attribute* is an attribute that divides the whole population into two groups (privileged & unprivileged) that have differences in terms of receiving benefits.

Every dataset in Table 1 has one or two such attributes. For example, in case of credit card application datasets, based on protected attribute "sex", "male" is privileged and "female" is unprivileged; in case of health datasets, based on protected attribute "age", "young" is privileged and "old" is unprivileged.

- *Group fairness* is the goal that based on the protected attribute, privileged and unprivileged groups will be treated similarly.
- *Individual fairness* means similar outcomes go to similar individuals.

Table 2 contains five performance metrics and four fairness metrics we used. These metrics are selected since they were widely used in the literature [5, 32, 43–45]. Prediction performance is measured in terms of *recall, false alarm, accuracy, precision, F1*; fairness is measured using *AOD, EOD, SPD, DI*. All of these can be calculated from the confusion matrix of binary classification containing four cells - true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN).

- For recall, accuracy, precision, F1 *larger values* are *better*;
- For false alarm, AOD, EOD, SPD *smaller values* are *better*.

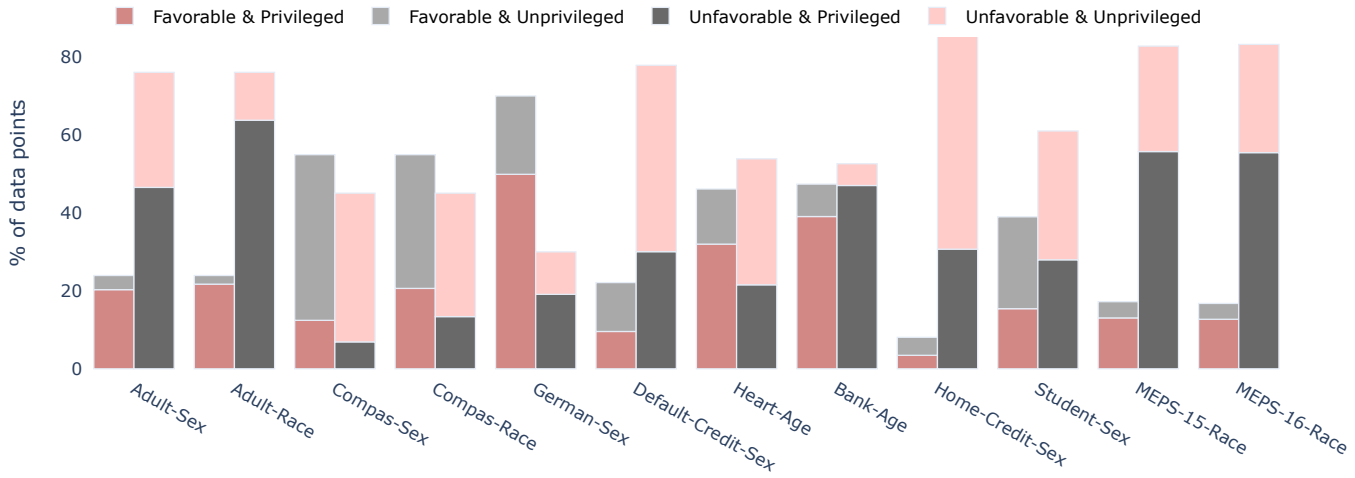

Figure 2: Most of the datasets showing not only class imbalance but also imbalance based on the protected attribute.

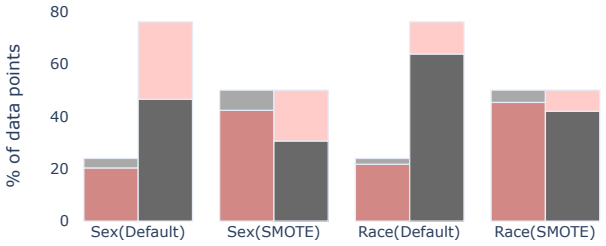

Figure 3: Effects of SMOTE class balancing technique on Adult dataset for two protected attributes "sex" and "race".

- DI is a ratio and there is no bias when value of DI is 1. For comprehensibility, while showing results we compute abs(1 - DI) so that all four fairness metrics are lower the better (0 means no bias).

Prior research [11, 30, 31, 46] has shown that classification models built from the datasets of Table 1 show bias. Why? What are the natures of these datasets that result in bias?

In this paper, we postulate that data has a history and that history introduces bias. For example, consider a team of humans labeling data as "risky loan applicant" or otherwise. If that team has any biases (conscious or otherwise) against certain social groups, then the bias of those humans introduces improper/unfair data labels.

Another way history can affect the data is *selection bias*. Suppose we are collecting package data relating to the kinds of items shipped via Amazon to a particular suburb. Suppose there is some institutional bias that tends to results in low annual incomes for persons in certain suburbs[2]. The delivery data collected from those suburbs would contain all the political and economic biases that tend to perpetuate lower socio-economic neighborhoods.

The next two sections explore the intuitions of the last two paragraphs in more detail. Specifically, we will look into data imbalance and improper data labeling. This in turn will lead to the Fair-SMOTE algorithm that deletes biased labels and balances all distributions between positive and negative examples.

[2]E.g. If politicians spend less money on schools in a poorer district; then that district has (a) fewer exceptional schools; (b) fewer graduates with job skills in for high-paying jobs in high demand; (c) consistently lower incomes from generation to generation.

3.1 Data Imbalance

When a classification model is trained on imbalanced data, the trained model shows bias against a certain group of people. We mention that such imbalances are quite common. Figure 2 displays distributions within our datasets. Note that, in most cases, we see a *class imbalance*; i.e the number of observations per class is not equally distributed. Further, we see that *the imbalance is not just based on class, but also on protected attribute.*

For example, consider the Adult dataset. Here we are predicting the annual income of a person. There are two classes. "High income" ($\geq$ \$50,000) which is *favorable label* and "low income" (< \$50,000) which is *unfavorable label*. The first grouped bar in Figure 2 has two bars grouped together. The first bar is for "high income" class (24%) and the second bar is for "low income" (76%) class. It is clear that the number of instances for "low income" is more than three times of instances for "high income". This is an example of class imbalance. Now, both the bars are subdivided based on *protected attribute* "sex" ("male" and "female"). For "high income" or *favorable label*, 86% instances are "male" and only 14% are female. For "low income" or *unfavorable label*, 60% instances are "male" and 40% are female. Overall, this dataset contains more examples of "male" (privileged) getting *favorable label* and female (unprivileged) getting *unfavorable label*.

Class imbalance is a widely studied topic in ML domain. There are mainly two ways of balancing imbalanced classes:

- Oversample the minority class;
- or undersample the majority class.

We want to see how various class balancing techniques affect fairness. In Table 3 we are showing the results for five most commonly used class balancing techniques on Adult dataset. One of them is undersampling (RUS- Random Under Sampling), other three (ROS-

Table 3: Effects of various class balancing techniques on Adult dataset (note: for the metrics with '+' more is better and for the metrics with '-' less is better). For each metric, cell with best score is highlighted.

	Recall(+)	False alarm(-)	Precision(+)	Accuracy(+)	F1 Score(+)	AOD(-)	EOD(-)	SPD(-)	DI(-)
Default	0.42	0.07	0.69	0.83	0.54	0.12	0.24	0.21	0.56
RUS	0.74	0.23	0.48	0.76	0.59	0.09	0.36	0.37	0.61
ROS	0.74	0.24	0.47	0.75	0.59	0.08	0.34	0.35	0.64
SMOTE	0.70	0.25	0.49	0.70	0.64	0.17	0.37	0.33	0.58
KMeans-SMOTE	0.73	0.24	0.48	0.74	0.58	0.08	0.35	0.36	0.62

Table 4: Percentage of data points failing situation testing for 10 datasets.

	Adult (Sex)	Adult (Race)	Compas (Sex)	Compas (Race)	German (Sex)	Default Credit (Sex)	Heart-Health (Age)	Bank Marketing (Age)	Home Credit (Sex)	Student (Sex)	MEPS-15 (Race)	MEPS-16 (Race)
% of Rows failed	11%	3%	18%	8%	6%	6%	8%	19%	17%	4%	4%	4%

Random Over Sampling, SMOTE- Synthetic Minority Over Sampling Technique [8] and KMeans-SMOTE [47]) are oversampling techniques. Table 3 shows values of nine metrics - first five of them are performance metrics and last four of them are fairness metrics. We used logistic regression model here.

The important observation here is all four of the class balancing techniques are increasing bias scores mean damaging fairness (lower is better here). To better understand this, see Figure 3. It gives a visualization of using SMOTE [8] on Adult dataset. SMOTE generates synthetic samples for minority class data points to equalize two classes. Suppose a data point from minority class is denoted as X where $x_1, x_2, .., x_n$ are the attributes and its nearest neighbor is $X^{'}$ ($x_1^{'}$,$x_2^{'}$,..,$x_n^{'}$). According to SMOTE, a new data point Y ($y_1, y_2, .., y_n$) is generated by the following formula:

$$Y = X + rand(0, 1) * (X - X^{'})$$

SMOTE definitely balances the majority class and minority class but it damages the protected attribute balance even more. Thus Figure 3 explains the results of Table 3.

For space reasons, we have shown this with one dataset and one technique (SMOTE) but in other datasets also, we got similar pattern. That said, traditional class balancing methods improve performance of the model but damage fairness (usually). The reason is *these techniques randomly generate/discard samples just to equalize two classes and completely ignore the attributes and hence damage the protected attribute balance even more.*

To fix this, we propose Fair-SMOTE. Like SMOTE, Fair-SMOTE will generate synthetic examples. But Fair-SMOTE takes much more care than SMOTE for exactly how it produces new examples. Specifically, it balances data based on class and sensitive attributes such that privileged and unprivileged groups have an equal amount of positive and negative examples in the training data (for more details, see §4).

3.2 Improper Data Labeling

Some prior works [35, 48, 49] argue that improper labeling could be a reason behind bias. We used the concept of *situation testing* to validate how labeling can affect fairness of the model. *Situation testing* is a research technique used in the legal field [6] where decision makers' candid responses to applicant's personal characteristics are captured and analyzed. For example:

- A pair of research assistants (a male and a female with almost equivalent qualities) undergo the same procedure, such as applying for a job.
- Now the treatments they get from the decision-maker are analyzed to find discrimination.

Situation testing as a legal tactic has been widely used both in the United States [6] and the European Union [7]. Luong et al. [50] first used the concept of situation testing in classification problems. They used K-NN approach to find out similar individuals getting different outcomes to find discrimination. Later, Zhang et al. [51] used causal bayesian networks to do the same. The core idea of our situation testing is much simpler:

- Flip the value of protected attribute for every data point.
- See whether prediction given by the model changes or not.

In our implementation, a logistic regression model is trained first and then all the data points are predicted. After that, the protected attribute value for every data point is flipped (e.g. male to female, white to non-white) and tested again to validate whether model prediction changes or not. If the result changes, that particular data point fails the situation testing.

Table 4 shows the median of ten runs for all the datasets. We see all the datasets more or less contain these kinds of biased data points. That means we have found *biased labels*. Note that, we use logistic regression model for *situation testing* but any other supervised model can be chosen. We picked logistic regression because it is a simple model and can be trained with a very low amount of data (compared to DL models) [52]. Choosing a different model may change the outcome a bit. We will explore that in future.

4 METHODOLOGY

Summarizing the above, we postulate that data imbalance and improper labeling are the two main reasons for model bias. Further, we assert that if we can solve these two problems then final outcome

will be a fairer model generating fairer prediction. This section describes experiments to test that idea.

4.1 Fair-SMOTE

Fair-SMOTE algorithm solves data imbalance. At first, the training data is divided into subgroups based on class and protected attribute. If class and protected attribute both are binary, then there will be 2*2 = 4 subgroups (Favorable & Privileged, Favorable & Unprivileged, Unfavorable & Privileged, Unfavorable & Unprivileged). Initially, these subgroups are of unequal sizes.

Fair-SMOTE synthetically generates new data points for all the subgroups except the subgroup having the maximum number of data points. As a result, all subgroups become of equal size (same with the maximum one).

As stated in the introduction, one danger with mutating data is that important associations between variables can be lost. Hence, in this work, we take care to mutate by extrapolating between the values seen in two neighboring examples. In that mutation process, Fair-SMOTE *extrapolates all the variables by the same amount*. That is, if there exists some average case association between the pre-mutated examples, then that association is preserved in the mutant.

For data generation, we use two hyperparameters "mutation amount" (f) and "crossover frequency" (cr) like *Differential Evolution* [53]. They both lie in the range [0, 1]. The first one denotes at which probability the new data point will be different from the parent point (0.8 means 80% of the times) and the latter one denotes how much different the new data point will be. We have tried with 0.2 (< 50% probability), 0.5 (= 50%), & 0.8 (> 50%) and got best results with 0.8.

Algorithm 1 describes the pseudocode of Fair-SMOTE. It starts with randomly selecting a data point (parent point p) from a subgroup. Then using K-nearest neighbor, two data points (c1, c2) are selected which are closest to p. Next, according to the algorithm described, a new data point is created. Separate logic is used for boolean, symbolic, and numeric columns.

Fair-SMOTE does not randomly create a new data point. Rather it creates a data point that is very close to the parent point. Thus the generated data points belong to the same data distribution. This process is repeated until all the subgroups become of similar size.

After applying Fair-SMOTE, the training data contains equal proportion of both classes and the protected attribute.

Algorithm 1: Pseudocode of Fair-SMOTE

```
Input: Dataset, Protected Attribute(p_attrs), Class Label(cl)
Output: Balanced Dataset
Def get_ngbr(Dataset, knn):
    rand_sample_id = random(0, len(Dataset))
    parent = Dataset[rand_sample_id]
    ngbr = knn.kneighbors(parent,2)
    c1, c2 = Dataset[ngbr[0]], Dataset[ngbr[1]]
    return parent, c1, c2
count_groups = get_count(Dataset, p_attrs, cl)
max_size = max(count_groups)
cr, f = 0.8, 0.8 (user can pick any value in [0,1])
for c in cl do
    for attr in p_attrs do
        sub_data = Dataset(cl=c  &  p_attrs=attr)
        sub_group_size = count_groups[c][attr]
        to_generate = max_size - sub_group_size
        knn = NearestNeighbors(sub_data)
        for i in range(to_generate) do
            p,c1,c2 = get_ngbr(sub_data, knn)
            new_candidate = []
            for col in parent_candidate.columns do
                if col is boolean then
                    if cr > random(0,1) then
                        new_val = random(p[col],c1[col],c2[col])
                    else
                        new_val = p[col]
                    new_candidate.add(new_val)
                else if col is String then
                    new_val = random(p[col],c1[col],c2[col])
                    new_candidate.add(new_val)
                else if col is Numeric then
                    if cr > random(0,1) then
                        new_val = p[col] + f*(c1[col]-c2[col])
                    else
                        new_val = p[col]
                    new_candidate.add(new_val)
        Dataset.add(new_candidate)
```

4.2 Fair Situation Testing

At first, we use Fair-SMOTE to balance the training data. Fair-SMOTE is an oversampling technique and thus increases the size of train set. We then use *situation testing* as mentioned in §3.2 to find out biased data points in the training data. We call this *situation testing* as *fair situation testing* because this is making the training data fairer. After finding biased data points, we remove them from training data. As Table 4 shows small percentage numbers, we do not lose much of the training data. We will show in the 'Results' section that this does not affect performance of the model much. After removal of biased data points, we train the model on the remaining training set and finally make the prediction on test data.

4.3 Experimental Design

Here we describe how we prepared the data for experiments to answer the research questions in §5. Our study used 10 datasets (Table 1) and 3 classification models - logistic regression (LSR), random forest (RF), and support vector machine (SVM). In fairness domain, datasets are not very large in size and also have small dimensions. That is why we see most of the prior works [5, 9, 31, 43] choose simple models like us instead of deep learning models. For every experiment, we split the datasets using 5 fold cross-validation (train - 80%, test - 20%) and repeat 10 times with random seeds and finally report the median. The rows containing missing values are ignored, continuous features are converted to categorical (e.g., age<25: young, age>=25: old), non-numerical features are converted to numerical (e.g., male: 1, female: 0), finally, all the feature values are normalized (converted between 0 to 1). These basic conversions are done for all the experiments. Classification model is first trained on training data and then tested on test data; we report the median of ten runs.

Figure 4 shows the block-diagram of one repeat of our framework.

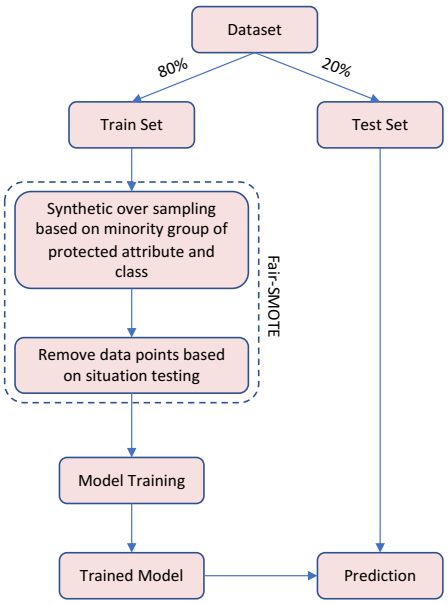

Figure 4: Block diagram of Fair-SMOTE

4.4 Statistical Tests

While comparing Fair-SMOTE with other techniques, we use Scott-Knott test [54, 55] to compare two distributions. Scott-Knott is a recursive bi-clustering algorithm that terminates when the difference between the two split groups is not significant. Scott-Knott searches for split points that maximize the expected value of the difference between the means of the two resulting groups. If a group l is split into two groups m and n, Scott-Knott searches for the split point that maximizes

$$E[\Delta] = |m|/|l|(E[m] - E[l])^2 + |n|/|l|(E[n] - E[l])^2$$

where $|m|$ is the size of group m. The result of the Scott-Knott test is ranks assigned to each result set; higher the rank, better the result. Scott-Knott ranks two results the same if the difference between the distributions is not significant.

5 RESULTS

The results are structured around five research questions.

RQ1. Can we find bias by just looking at the training data?

In §3 we said a machine learning model acquires bias from training data. RQ1 asks for the signals we must see to identify whether training data has bias or not. That is an important question to ask because if that is doable and that bias can be removed before model training, then the chances of bias affecting the final decision reduce significantly. Table 5 shows results for three different models (logistic regression (LSR), random forest (RF), support vector machine (SVM)), and Table 6 shows results for one model (LSR) only. The row "Default" is when we train the model on raw data. Results show that "Default" row has significantly high bias scores for all the datasets that means trained model is showing discrimination. Previously we have dived deep into these datasets to find reasons for bias. Here we are summarizing the sanity checks every dataset should go through before model training to avoid discrimination.

- **Data distribution** - The training data should be almost balanced based on class and protected attribute.
- **Data label** - Every data point should go through *situation testing* to see whether label has bias or not.

If we find bias in the training data, we should apply Fair-SMOTE to remove that and get fair outcomes. Thus, the answer for RQ1 is "**Yes, we can find bias by just looking at the training data**"

RQ2. Are standard class balancing techniques helpful to reduce bias?

For answering RQ2, we chose the most used class balancing technique, which is SMOTE [8]. Table 5 contains results for three datasets and three different learners (LSR, RF, SVM). In that table, for a particular dataset and for a particular performance metric:

- Cells with darker backgrounds denote treatments that are performing *better* than anything else.
- Conversely, cells with a white background denote treatments that are performing *worse* than anything else;.

SMOTE consistently increases bias scores (AOD, EOD, SPD, DI) mean damaging fairness but performs similar/better than "Default" in case of performance metrics (as measured by recall, false alarm, precision, accuracy & F1). Thus, the answer for RQ2 is "**No, standard class balancing techniques are not helpful since, in their enthusiasm to optimize model performance, they seem to also amplify model bias.**"

RQ3. Can Fair-SMOTE reduce bias?

Table 5 answers this question. Looking at the bias metrics (AOD, EOD, SPD, DI), Fair-SMOTE significantly reduces all the bias scores mean increasing fairness (see the darker colored cells).

As to the performance measures (recall, false alarm, precision, accuracy, and F1) it is hard to get a visual summary of the results just by looking at Table 5. For that purpose, we turn to rows 1,2,3,4 of Table 7. Based on Scott-Knott tests of §4.4, these rows count the number of times Fair-SMOTE wins, losses or ties compared to SMOTE (here, "tie" means "is assigned the same rank by Scott-Knott"). Those counts, in rows 1,2,3,4 confirm the visual patterns of Table 5:

- As to the performance measures (recall, false alarm, precision, accuracy, and F1), these methods often tie.
- But looking at the highlighted bias metrics results for AOD, EOD, SPD, & DI, Fair-SMOTE is clearly performing better than SMOTE.

Thus, the answer for RQ3 is "**Yes, Fair-SMOTE reduces bias significantly and performs much better than SMOTE.**"

RQ4. How well does Fair-SMOTE perform compared to the state of the art bias mitigation algorithms?

Table 7 compares Fair-SMOTE against other tools that try to find and fix bias. "Default" shows the off-the-shelf learners; "Fairway"

Table 5: Results for RQ1, RQ2 & RQ3. In this table "Default" means off-the-shelf learner; SMOTE is an algorithm by Chawla et al. [8] from 2002; and Fair-SMOTE is the algorithm introduced by this paper. Cells show medians for 10 runs. Here, the darkest cells show top rank (note: for the metrics with '+' more is better and for the metrics with '-' less is better). The lighter and lightest cells show rank two and rank three respectively; the white cells show the worst rank. Rankings were calculated via Scott-Knott test (§4.4).

Dataset	Protected Attribute	Algorithms	Recall (+)	False alarm (-)	Precision (+)	Accuracy (+)	F1 Score (+)	AOD (-)	EOD (-)	SPD (-)	DI (-)
Adult Census Income	Sex	Default - LSR	0.42	0.07	0.69	0.83	0.54	0.12	0.24	0.21	0.56
		Default - RF	0.51	0.06	0.72	0.83	0.59	0.09	0.17	0.17	0.33
		Default - SVM	0.35	0.04	0.81	0.82	0.49	0.1	0.24	0.14	0.43
		SMOTE - LSR	0.70	0.25	0.49	0.70	0.64	0.17	0.37	0.33	0.58
		SMOTE - RF	0.61	0.07	0.71	0.83	0.62	0.08	0.16	0.16	0.32
		SMOTE - SVM	0.71	0.19	0.55	0.78	0.62	0.11	0.52	0.42	0.46
		Fair-SMOTE - LSR	0.71	0.25	0.51	0.73	0.62	0.01	0.02	0.03	0.15
		Fair-SMOTE - RF	0.69	0.2	0.53	0.78	0.6	0.03	0.04	0.1	0.22
		Fair-SMOTE - SVM	0.73	0.23	0.51	0.76	0.6	0.02	0.02	0.08	0.21
Adult Census Income	Race	Default - LSR	0.42	0.05	0.69	0.81	0.52	0.06	0.15	0.16	0.52
		Default - RF	0.53	0.07	0.7	0.83	0.6	0.12	0.16	0.12	0.57
		Default - SVM	0.35	0.03	0.8	0.82	0.49	0.08	0.11	0.10	0.35
		SMOTE - LSR	0.71	0.23	0.49	0.72	0.61	0.16	0.19	0.23	0.56
		SMOTE - RF	0.66	0.08	0.67	0.83	0.63	0.09	0.12	0.12	0.52
		SMOTE - SVM	0.58	0.11	0.62	0.81	0.6	0.09	0.12	0.17	0.32
		Fair-SMOTE - LSR	0.7	0.22	0.51	0.72	0.62	0.04	0.03	0.05	0.26
		Fair-SMOTE - RF	0.73	0.2	0.52	0.8	0.61	0.01	0.02	0.08	0.29
		Fair-SMOTE - SVM	0.71	0.26	0.5	0.75	0.61	0.01	0.01	0.06	0.18
Compas	Sex	Default - LSR	0.73	0.38	0.66	0.64	0.61	0.05	0.14	0.18	0.24
		Default - RF	0.75	0.45	0.66	0.67	0.72	0.11	0.18	0.2	0.34
		Default - SVM	0.77	0.45	0.66	0.67	0.71	0.1	0.15	0.22	0.28
		SMOTE - LSR	0.65	0.33	0.62	0.6	0.65	0.08	0.19	0.22	0.31
		SMOTE - RF	0.72	0.42	0.67	0.65	0.7	0.11	0.22	0.26	0.3
		SMOTE - SVM	0.7	0.36	0.68	0.66	0.69	0.1	0.21	0.31	0.38
		Fair-SMOTE - LSR	0.62	0.32	0.56	0.55	0.65	0.02	0.05	0.08	0.04
		Fair-SMOTE - RF	0.71	0.44	0.66	0.65	0.7	0.04	0.03	0.1	0.02
		Fair-SMOTE - SVM	0.79	0.5	0.65	0.66	0.71	0.02	0.01	0.06	0.08
Compas	Race	Default - LSR	0.69	0.39	0.65	0.64	0.68	0.05	0.11	0.12	0.21
		Default - RF	0.75	0.44	0.66	0.66	0.7	0.07	0.17	0.21	0.24
		Default - SVM	0.77	0.45	0.66	0.67	0.71	0.07	0.14	0.18	0.24
		SMOTE - LSR	0.61	0.32	0.61	0.6	0.63	0.06	0.16	0.14	0.27
		SMOTE - RF	0.75	0.42	0.67	0.66	0.7	0.07	0.13	0.19	0.31
		SMOTE - SVM	0.7	0.39	0.68	0.66	0.69	0.09	0.12	0.16	0.24
		Fair-SMOTE - LSR	0.62	0.30	0.56	0.55	0.66	0.01	0.05	0.06	0.11
		Fair-SMOTE - RF	0.66	0.39	0.67	0.65	0.67	0.01	0.03	0.02	0.10
		Fair-SMOTE - SVM	0.7	0.41	0.67	0.65	0.68	0.02	0.06	0.08	0.12
MEPS - 16	Race	Default - LSR	0.35	0.05	0.65	0.85	0.44	0.04	0.1	0.07	0.43
		Default - RF	0.38	0.11	0.62	0.81	0.41	0.08	0.12	0.08	0.36
		Default - SVM	0.31	0.08	0.66	0.79	0.4	0.09	0.1	0.09	0.32
		SMOTE - LSR	0.65	0.22	0.58	0.78	0.49	0.17	0.11	0.11	0.49
		SMOTE - RF	0.63	0.18	0.56	0.78	0.52	0.12	0.19	0.13	0.31
		SMOTE - SVM	0.62	0.23	0.49	0.79	0.5	0.13	0.18	0.15	0.47
		Fair-SMOTE - LSR	0.66	0.2	0.41	0.77	0.51	0.01	0.03	0.04	0.17
		Fair-SMOTE - RF	0.62	0.21	0.38	0.77	0.48	0.03	0.04	0.03	0.19
		Fair-SMOTE - SVM	0.61	0.18	0.39	0.76	0.49	0.04	0.03	0.05	0.19

is the Chakraborty et al. [5] system from FSE'20; and OP is the Optimized Pre-processing method from NIPS'17 [9]. Here, the learner is logistic regression since, for performance measures, LSR has best results in the Fair-SMOTE results of Table 5.

Rows 5,6,7,8,9,10,11,12 of Table 7 summarize these results. Measured in terms of bias reduction, all the methods often tie. But observing the highlighted cells in that table, we see that the Fair-SMOTE performs much better (in terms of recall and F1) than anything else.

Thus, the answer for RQ4 is "**Fair-SMOTE performs similar or better than two state of the art bias mitigation algorithms in case of fairness and consistently gives higher recall and F1 score.**" That means we do not have to compromise performance anymore to achieve fairness. Fair-SMOTE is able to provide both - better fairness and performance. This is the biggest achievement of this work.

RQ5. Can Fair-SMOTE reduce bias for more than one protected attribute?

In the literature, we are unaware of any work trying to reduce bias for more than one protected attribute at a time. Typically what researchers do is try to eliminate bias based on one protected attribute, one at a time. We experimented on Adult dataset having two protected attributes (sex, race). The idea is to balance the training data with respect to class and two protected attributes. That means we need to find out among the eight (2^3) possible subgroups which

Table 6: Results for RQ3, RQ4 (learner= Logistic Regression). In this table. "Default" denotes off-the-shelf logistic regression; OP is Calmon et al.'s system from NIPS'17 [9]; Fairway is Chakraborty et al.'s system from FSE'20 [5]; and Fair-SMOTE is the algorithm introduced by this paper. Cells show medians for 10 runs. Here, the darker cells show top rank (note: for the metrics with '+' more is better and for the metrics with '-' less is better). The lighter cells show rank two; white shows lowest rank (worst performance). Rankings were calculated via the Scott-Knott test (§4.4)

Dataset	Protected Attribute	Algorithms	Recall (+)	False alarm (-)	Precision (+)	Accuracy (+)	F1 Score (+)	AOD (-)	EOD (-)	SPD (-)	DI (-)
Adult Census Income	Sex	Default	0.42	0.07	0.69	0.83	0.54	0.12	0.24	0.21	0.56
		OP	0.41	0.09	0.61	0.76	0.51	0.04	0.03	0.04	0.14
		Fairway	0.25	0.04	0.70	0.72	0.42	0.02	0.03	0.01	0.11
		Fair SMOTE	0.71	0.25	0.51	0.73	0.62	0.01	0.02	0.03	0.12
Adult Census Income	Race	Default	0.42	0.05	0.69	0.81	0.52	0.06	0.15	0.16	0.52
		OP	0.38	0.06	0.66	0.78	0.48	0.03	0.02	0.05	0.21
		Fairway	0.36	0.04	0.70	0.73	0.44	0.02	0.03	0.06	0.32
		Fair SMOTE	0.7	0.22	0.51	0.72	0.62	0.04	0.03	0.05	0.26
Compas	Sex	Default	0.73	0.38	0.66	0.64	0.61	0.05	0.14	0.18	0.28
		OP	0.71	0.36	0.64	0.62	0.60	0.04	0.05	0.06	0.09
		Fairway	0.56	0.22	0.57	0.58	0.58	0.03	0.03	0.06	0.08
		Fair SMOTE	0.62	0.32	0.56	0.55	0.65	0.02	0.05	0.08	0.09
Compas	Race	Default	0.69	0.39	0.65	0.64	0.68	0.05	0.11	0.12	0.21
		OP	0.68	0.33	0.63	0.62	0.67	0.03	0.06	0.06	0.12
		Fairway	0.55	0.21	0.58	0.58	0.56	0.02	0.04	0.07	0.07
		Fair SMOTE	0.62	0.30	0.56	0.55	0.66	0.01	0.05	0.06	0.11
German Credit	Sex	Default	0.94	0.81	0.72	0.76	0.82	0.11	0.08	0.14	0.15
		OP	0.75	0.73	0.71	0.73	0.71	0.04	0.05	0.06	0.12
		Fairway	0.78	0.76	0.68	0.65	0.73	0.05	0.04	0.07	0.11
		Fair SMOTE	0.62	0.36	0.71	0.64	0.71	0.05	0.05	0.05	0.13
Default Credit	Sex	Default	0.25	0.07	0.7	0.78	0.34	0.05	0.08	0.06	0.45
		OP	0.28	0.06	0.65	0.70	0.32	0.01	0.02	0.03	0.09
		Fairway	0.21	0.04	0.67	0.67	0.33	0.01	0.04	0.03	0.12
		Fair SMOTE	0.58	0.26	0.39	0.68	0.44	0.02	0.03	0.05	0.03
Heart Health	Age	Default	0.72	0.2	0.74	0.72	0.74	0.11	0.13	0.32	0.44
		OP	0.69	0.19	0.72	0.69	0.69	0.04	0.06	0.09	0.14
		Fairway	0.65	0.21	0.69	0.67	0.68	0.05	0.05	0.04	0.18
		Fair SMOTE	0.66	0.20	0.69	0.68	0.66	0.08	0.07	0.08	0.20
Bank Marketing	Age	Default	0.73	0.21	0.76	0.77	0.77	0.14	0.22	0.24	0.21
		OP	0.72	0.20	0.74	0.75	0.75	0.05	0.05	0.02	0.04
		Fairway	0.71	0.17	0.73	0.71	0.73	0.04	0.03	0.03	0.05
		Fair SMOTE	0.76	0.18	0.72	0.72	0.74	0.05	0.07	0.05	0.03
Home Credit	Sex	Default	0.31	0.18	0.28	0.86	0.29	0.07	0.06	0.08	0.59
		OP	0.3	0.17	0.30	0.83	0.31	0.02	0.04	0.04	0.16
		Fairway	0.28	0.12	0.25	0.65	0.26	0.02	0.04	0.03	013
		Fair SMOTE	0.33	0.18	0.31	0.75	0.32	0.03	0.02	0.03	0.15
Student Performance	Sex	Default	0.81	0.06	0.85	0.88	0.83	0.06	0.05	0.06	0.12
		OP	0.79	0.06	0.83	0.83	0.82	0.03	0.02	0.03	0.06
		Fairway	0.76	0.05	0.81	0.84	0.84	0.03	0.02	0.04	0.07
		Fair SMOTE	0.91	0.10	0.84	0.87	0.86	0.04	0.04	0.04	0.08
MEPS - 15	Race	Default	0.36	0.03	0.68	0.85	0.45	0.04	0.05	0.08	0.36
		OP	0.35	0.04	0.66	0.83	0.44	0.04	0.02	0.06	0.15
		Fairway	0.35	0.04	0.42	0.77	0.41	0.03	0.02	0.04	0.12
		Fair SMOTE	0.68	0.22	0.41	0.77	0.53	0.02	0.02	0.05	0.15
MEPS - 16	Sex	Default	0.35	0.05	0.65	0.85	0.44	0.04	0.1	0.07	0.43
		OP	0.34	0.04	0.65	0.83	0.44	0.05	0.02	0.05	0.12
		Fairway	0.32	0.04	0.55	0.76	0.42	0.03	0.02	0.04	0.16
		Fair SMOTE	0.66	0.2	0.41	0.77	0.51	0.01	0.03	0.04	0.17

one has the most number of data points. Then we need to make other subgroups of the same size with the one having most number of data points. That is done by generating new data points using Fair-SMOTE. Table 8 shows results that bias is reduced for both the attributes along with higher recall and F1 score. Hence, the answer of RQ5 is "**Yes, Fair-SMOTE can simultaneously reduce bias for more than one protected attribute.**"

6 DISCUSSION: WHY FAIR-SMOTE?

Here we discuss what makes Fair-SMOTE unique and more useful than prior works in the fairness domain.

Combination: This is only the second SE work (after Fairway [5]) in fairness domain where primary focus is bias mitigation. There are a lot of papers in ML domain where various techniques are provided to remove bias from model behavior. Still, when it comes to applicability, we see software practitioners find ML fairness as a complicated topic. Because finding bias, explaining bias, and removing bias have been treated as separate problems and thus created more confusion. That's where this work makes a significant contribution by combining all of them together. We first find data imbalance and improperly labeled data points (by situation testing) and then use oversampling to balance the data and remove

Table 7: RQ3, RQ4 results. Summarized information of comparing Fair-SMOTE with SMOTE[8], Fairway[5] & Optimized Pre-processing[9] based on results of 10 datasets and 3 learners (LSR, RF, SVM). Number of wins, ties, and losses are calculated based on Scott-Knott ranks for each metric. Highlighted cells show Fair-SMOTE significantly outperforming others.

		Recall	False alarm	Precision	Accuracy	F1 Score	AOD	EOD	SPD	DI	Total
					SMOTE vs Fair-SMOTE						
1	**Win**	4	4	1	6	3	33	33	34	32	150
2	**Tie**	25	27	29	28	30	2	3	2	2	148
3	**Loss**	7	5	6	2	3	1	0	0	2	26
4	**Win + Tie**	29	31	30	34	33	35	36	36	34	298/324
					Fairway vs Fair-SMOTE						
5	**Win**	14	4	6	5	20	3	2	3	4	61
6	**Tie**	20	20	27	28	14	30	31	32	31	233
7	**Loss**	2	12	3	3	2	3	3	1	1	28
8	**Win + Tie**	34	24	33	33	34	33	33	35	35	294/324
					Optimized Pre-processing vs Fair-SMOTE						
9	**Win**	10	7	4	3	12	1	2	2	3	44
10	**Tie**	21	22	26	30	20	34	33	32	31	249
11	**Loss**	5	7	6	3	4	1	1	2	2	31
12	**Win + Tie**	31	29	30	33	32	35	35	34	34	293/324

Table 8: RQ5 results. Fair-SMOTE reducing bias for 'sex' and 'race' simultaneously (Adult dataset). Best cells are highlighted.

	Protected attribute	Recall(+)	False alarm(-)	Precision(+)	Accuracy(+)	F1 Score(+)	AOD(-)	EOD(-)	SPD(-)	DI(-)
Default	Sex	0.42	0.07	0.69	0.83	0.52	0.12	0.24	0.21	0.56
	Race						0.06	0.15	0.16	0.52
Fair-SMOTE	Sex	0.71	0.24	0.49	0.75	0.59	0.02	0.05	0.09	0.27
	Race						0.01	0.03	0.08	0.22

improperly labeled data points. As an outcome, we generate fair results.

Uncompromising: Our framework improves fairness scores along with F1 score and recall. It does not damage accuracy and precision much also. That said, unlike much prior work [9–11, 30–32], we can do bias mitigation *without* compromising predictive performance. We attribute our success in this regard to our sampling policy. None of our mutators damage the associations between attributes. Rather, we just carefully resample the data to avoid certain hard cases (where the training data can only see a few examples of each kind of row).

Group & Individual: Our data balancing approach takes care of *group fairness* where goal is based on the protected attribute, privileged and unprivileged groups will be treated similarly. Our situation testing method takes care of *individual fairness* where goal is to provide similar outcomes to similar individuals.

Generality: We entirely focused on data to find and remove bias. There are works where optimization tricks have been used while model training to remove bias [10, 31]. These works are model specific and most of the time combine with internal model logic. However, Fair-SMOTE does not require access to inside model logic. Thus it is much more general as it can be used for any kind of model. In this work, we used three simple models and got promising results. In future, we will explore deep learning models.

Versatility: We used Fair-SMOTE for only classification datasets here. However, the core idea of Fair-SMOTE is keeping equal proportion of all the protected groups in the training data. We believe the same approach can be applied to regression problems. In future we would like to explore that. Besides, the same approach can be applied for image data (face recognition) to train model with equal proportion of white and black faces so that model does not show racial bias. That means Fair-SMOTE can be easily adopted by other domains to solve bias issues caused by data imbalance.

7 THREATS TO VALIDITY

Sampling Bias - As per our knowledge, this is the most extensive fairness study using 10 real-world datasets and 3 classification models. Still, conclusions may change a bit if other datasets and models are used.

Evaluation Bias - We used the four most popular fairness metrics in this study. Prior works [5, 31, 32] only used two or three metrics although IBM AIF360 contains more than 50 metrics. In future, we will explore more evaluation criteria.

Internal Validity - Where prior researchers [37, 38] focused on attributes to find causes of bias, we concentrated on data distribution and labels. However, there could be some other reasons. A recent Amazon paper comments on some other reasons such as objective function bias, homogenization bias [48]. We could not validate these biases in our datasets as it was out of scope. In future, we would like to explore those reasons if industry datasets become available.

External Validity - This work is based on binary classification and tabular data which are very common in AI software. We are currently working on extending it to regression models. In future work, we would extend this work to other domains such as text mining and image processing.

8 CONCLUSION

This paper has tested the Fair-SMOTE tactic for mitigating bias in ML software. Fair-SMOTE assumes the root causes of bias are the prior decisions that control (a) what data was collected and (b) the labels assigned to the data. Fair-SMOTE:

> *(A) Removes biased labels; and (B) rebalances internal distributions such that they are equal based on class and sensitive attributes.*

As seen above, Fair-SMOTE was just as effective at bias mitigation as two other state-of-the-art algorithms [5, 9] and more effective in terms of achieving higher performance (measured in terms of recall and F1). Also, Fair-SMOTE runs 220% faster (median value across ten data sets) than Chakraborty et.al [5].

Based on the above, we offer three conclusions:

(1) We can recommend Fair-SMOTE for bias mitigation.
(2) We can reject the pessimism of Berk et al. [33] who, previously, had been worried that the cost of fairness was a reduction in learner performance.
(3) More generally, rather than blindly applying some optimization methods it can be better to:
   - Reflect on the domain;
   - Use insights from that reflection to guide improvements in that domain.

ACKNOWLEDGEMENTS

The work was partially funded by LAS and NSF grant #1908762.

REFERENCES

[1] "Amazon scraps secret ai recruiting tool that showed bias against women," Oct 2018. [Online]. Available: https://www.reuters.com/article/us-amazon-com-jobs-automation-insight/amazon-scraps-secret-ai-recruiting-tool-that-showed-bias-against-women-idUSKCN1MK08G
[2] "Study finds gender and skin-type bias in commercial artificial-intelligence systems," 2018. [Online]. Available: http://news.mit.edu/2018/study-finds-gender-skin-type-bias-artificial-intelligence-systems-0212
[3] A. Caliskan, J. J. Bryson, and A. Narayanan, "Semantics derived automatically from language corpora contain human-like biases," *Science*, vol. 356, no. 6334, pp. 183–186, 2017. [Online]. Available: https://science.sciencemag.org/content/356/6334/183
[4] B. Johnson, J. Bartola, R. Angell, K. Keith, S. Witty, S. J. Giguere, and Y. Brun, "Fairkit, fairkit, on the wall, who's the fairest of them all? supporting data scientists in training fair models," 2020. [Online]. Available: https://arxiv.org/abs/2012.09951
[5] J. Chakraborty, S. Majumder, Z. Yu, and T. Menzies, "Fairway: A way to build fair ml software," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 654–665. [Online]. Available: https://doi.org/10.1145/3368089.3409697
[6] "Situation testing for employment discrimination in the united states of america," 2007. [Online]. Available: https://www.cairn.info/revue-horizons-strategiques-2007-3-page-17.htm
[7] "Proving discrimination cases – the role of situation testing," 2009. [Online]. Available: https://www.migpolgroup.com/_old/portfolio/proving-discrimination-cases-the-role-of-situation-testing/
[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, p. 321–357, Jun 2002. [Online]. Available: http://dx.doi.org/10.1613/jair.953
[9] F. Calmon, D. Wei, B. Vinzamuri, K. Natesan Ramamurthy, and K. R. Varshney, "Optimized pre-processing for discrimination prevention," in *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Curran Associates, Inc., 2017, pp. 3992–4001. [Online]. Available: http://papers.nips.cc/paper/6988-optimized-pre-processing-for-discrimination-prevention.pdf
[10] B. H. Zhang, B. Lemoine, and M. Mitchell, "Mitigating unwanted biases with adversarial learning," 2018. [Online]. Available: http://www.aies-conference.com/wp-content/papers/main/AIES_2018_paper_162.pdf
[11] F. Kamiran, S. Mansha, A. Karim, and X. Zhang, "Exploiting reject option in classification for social discrimination control," *Inf. Sci.*, vol. 425, no. C, p. 18–33, Jan. 2018. [Online]. Available: https://doi.org/10.1016/j.ins.2017.09.064
[12] "Fairware 2018:international workshop on software fairness." [Online]. Available: http://fairware.cs.umass.edu/
[13] "Explain 2019." [Online]. Available: https://2019.ase-conferences.org/home/explain-2019
[14] "Ai fairness 360: An extensible toolkit for detecting, understanding, and mitigating unwanted algorithmic bias," 10 2018. [Online]. Available: https://github.com/IBM/AIF360
[15] "Fate: Fairness, accountability, transparency, and ethics in ai," 2018. [Online]. Available: https://www.microsoft.com/en-us/research/group/fate/
[16] "Facebook says it has a tool to detect bias in its artificial intelligence," 2018. [Online]. Available: https://qz.com/1268520/facebook-says-it-has-a-tool-to-detect-bias-in-its-artificial-intelligence/
[17] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: http://jmlr.org/papers/v13/bergstra12a.html
[18] "Uci:adult data set," 1994. [Online]. Available: http://mlr.cs.umass.edu/ml/datasets/Adult
[19] "propublica/compas-analysis," 2015. [Online]. Available: https://github.com/propublica/compas-analysis
[20] "Uci:statlog (german credit data) data set," 2000. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Statlog+(German+Credit+Data)
[21] "Uci:default of credit card clients data set," 2016. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients
[22] "Uci:heart disease data set," 2001. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Heart+Disease
[23] "Bank marketing uci," 2017. [Online]. Available: https://www.kaggle.com/c/bank-marketing-uci
[24] "Thome credit default risk," 2017. [Online]. Available: https://www.kaggle.com/c/home-credit-default-risk
[25] "Student performance data set," 2014. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Student+Performance
[26] "Medical expenditure panel survey," 2015. [Online]. Available: https://meps.ahrq.gov/mepsweb/
[27] R. Bellamy, K. Dey, M. Hind, S. C. Hoffman, S. Houde, K. Kannan, P. Lohia, J. Martino, S. Mehta, A. Mojsilovic, S. Nagar, K. Natesan Ramamurthy, J. Richards, D. Saha, P. Sattigeri, M. Singh, R. Kush, and Y. Zhang, "Ai fairness 360: An extensible toolkit for detecting, understanding, and mitigating unwanted algorithmic bias," 10 2018.
[28] T. Calders and S. Verwer, "Three naive bayes approaches for discrimination-free classification," *Data Min. Knowl. Discov.*, vol. 21, no. 2, p. 277–292, Sep. 2010. [Online]. Available: https://doi.org/10.1007/s10618-010-0190-x
[29] M. Feldman, S. Friedler, J. Moeller, C. Scheidegger, and S. Venkatasubramanian, "Certifying and removing disparate impact," 2015.
[30] F. Kamiran and T. Calders, "Data preprocessing techniques for classification without discrimination," *Knowledge and Information Systems*, vol. 33, no. 1, pp. 1–33, Oct 2012. [Online]. Available: https://doi.org/10.1007/s10115-011-0463-8
[31] T. Kamishima, S. Akaho, H. Asoh, and J. Sakuma, "Fairness-aware classifier with prejudice remover regularizer," in *Machine Learning and Knowledge Discovery in Databases*, P. A. Flach, T. De Bie, and N. Cristianini, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 35–50.
[32] M. Hardt, E. Price, and N. Srebro, "Equality of opportunity in supervised learning," 2016. [Online]. Available: https://arxiv.org/abs/1610.02413
[33] R. Berk, H. Heidari, S. Jabbari, M. Kearns, and A. Roth, "Fairness in criminal justice risk assessments: The state of the art," 2017.
[34] S. Maity, D. Mukherjee, M. Yurochkin, and Y. Sun, "There is no trade-off: enforcing fairness can improve accuracy," 2020. [Online]. Available: https://arxiv.org/abs/2011.03173
[35] H. Jiang and O. Nachum, "Identifying and correcting label bias in machine learning," 2019. [Online]. Available: https://arxiv.org/abs/1901.04966
[36] S. Yan, H.-t. Kao, and E. Ferrara, "Fair class balancing: Enhancing model fairness without observing sensitive attributes," in *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, ser. CIKM '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1715–1724.
[37] L. Zhang, Y. Wu, and X. Wu, "A causal framework for discovering and removing direct and indirect discrimination," 2016. [Online]. Available: https://www.ijcai.org/proceedings/2017/0549.pdf
[38] J. R. Loftus, C. Russell, M. J. Kusner, and R. Silva, "Causal reasoning for algorithmic fairness," 2018. [Online]. Available: https://arxiv.org/abs/1805.05859
[39] S. Galhotra, Y. Brun, and A. Meliou, "Fairness testing: testing software for discrimination," *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering - ESEC/FSE 2017*, 2017. [Online]. Available: http://dx.doi.org/10.1145/3106237.3106277
[40] S. Udeshi, P. Arora, and S. Chattopadhyay, "Automated directed fairness testing," *Proceedings of the 33rd ACM/IEEE International Conference on*

Automated Software Engineering - ASE 2018, 2018. [Online]. Available: http://dx.doi.org/10.1145/3238147.3238165

[41] A. Aggarwal, P. Lohia, S. Nagar, K. Dey, and D. Saha, "Black box fairness testing of machine learning models," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019. New York, NY, USA: ACM, 2019, pp. 625–635. [Online]. Available: http://doi.acm.org/10.1145/3338906.3338937

[42] P. Zhang, J. Wang, J. Sun, X. Dong, J. S. Dong, and T. Dai, "White-box fairness testing through adversarial sampling," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 949–960. [Online]. Available: https://doi.org/10.1145/3377811.3380331

[43] S. Biswas and H. Rajan, "Do the machine learning models on a crowd sourced platform exhibit bias? an empirical study on model fairness," *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Nov 2020. [Online]. Available: http://dx.doi.org/10.1145/3368089.3409704

[44] J. Chakraborty, T. Xia, F. M. Fahid, and T. Menzies, "Software engineering for fairness: A case study with hyperparameter optimization," 2019. [Online]. Available: https://arxiv.org/abs/1905.05786

[45] J. Chakraborty, K. Peng, and T. Menzies, "Making fair ml software using trustworthy explanation," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1229–1233. [Online]. Available: https://doi.org/10.1145/3324884.3418932

[46] R. Angell, B. Johnson, Y. Brun, and A. Meliou, "Themis: Automatically testing software for discrimination," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2018. New York, NY, USA: Association for Computing Machinery, 2018, p. 871–875. [Online]. Available: https://doi.org/10.1145/3236024.3264590

[47] G. Douzas, F. Bacao, and F. Last, "Improving imbalanced learning through a heuristic oversampling method based on k-means and smote," *Information Sciences*, vol. 465, p. 1–20, Oct 2018.

[48] S. Das and M. Donini, "Fairness measures for machine learning in finance," 2020. [Online]. Available: https://pages.awscloud.com/rs/112-TZM-766/images/Fairness.Measures.for.Machine.Learning.in.Finance.pdf

[49] T. Simons, "Addressing issues of fairness and bias in ai," 2020. [Online]. Available: https://blogs.thomsonreuters.com/answerson/ai-fairness-bias/

[50] B. T. Luong, S. Ruggieri, and F. Turini, "K-nn as an implementation of situation testing for discrimination discovery and prevention," in *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 502–510. [Online]. Available: https://doi.org/10.1145/2020408.2020488

[51] L. Zhang, Y. Wu, and X. Wu, "Situation testing-based discrimination discovery: A causal inference approach," in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, ser. IJCAI'16. AAAI Press, 2016, p. 2718–2724.

[52] M. A. Bujang, N. Sa'at, T. M. I. T. A. B. Sidik, and L. C. Joo, "Sample size guidelines for logistic regression from observational studies with large population: Emphasis on the accuracy between statistics and parameters based on real life clinical data," *The Malaysian Journal of Medical Sciences : MJMS*, vol. 25, pp. 122 – 130, 2018.

[53] R. Storn and K. Price, "Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, pp. 341–359, 01 1997.

[54] B. Ghotra, S. McIntosh, and A. E. Hassan, "Revisiting the impact of classification techniques on the performance of defect prediction models," in *2015 IEEE/ACM 37th IEEE ICSE*, vol. 1, 2015. [Online]. Available: https://ieeexplore.ieee.org/document/7194626

[55] N. Mittas and L. Angelis, "Ranking and clustering software cost estimation models through a multiple comparisons algorithm," *IEEE TSE*, vol. 39, no. 4, 2013. [Online]. Available: https://ieeexplore.ieee.org/document/6235961